

Decoupling Workload Admission from Policy Materialization in Kubernetes

*AI-generated draft written in the voice of the three authors
Non-submission exercise; Traditional Research framing*

Editorial notice. This document is an AI-generated writing exercise produced by three agents impersonating the paper’s authors. It is not a submission and is not text written by Antonio Ojea, Michael Aspinwall, or Dan Winship. Every number is traceable to the archived artifacts listed in Appendix A; where evidence is missing the text says so.

Abstract

NetworkPolicy binds permissions to labels, and labels to Pods that are created and replaced continuously. Engines that materialize an entry per peer identity at each endpoint carry state proportional to endpoints times identities, regardless of who communicates. kube-network-policies (KNP) places the work elsewhere. nftables steers uncached traffic of policy-selected Pods to NFQUEUE, a userspace evaluator decides when a connection is attempted, and an accepted verdict is stored as a conntrack label so the kernel forwards the rest of the connection. Local Pod addresses arrive through the runtime’s NRI hooks rather than the API server. We compare KNP with Cilium on a 720-worker cluster creating 35,000 Pods in a bidirectional-mesh identity sweep whose sandboxes block until a permitted TCP connection succeeds, so post-scheduling latency bounds time to first permitted communication. KNP completes the 35,000-identity mesh tier that Cilium’s producer excluded on predicted policy-map capacity (all Pods Running, 421 s P99, two startup-SLO failures). At 700 and 3,500 identities its P99 is 5.33 and 2.79 s, within 2.4 s of Cilium, at 2.5–3× lower achieved Pod throughput. At 7 identities with 5,000 Pods per ReplicaSet its P99 is 151 s against Cilium’s 6.8 s, unexplained. Each tier has one trial and no agent metrics. We show where the eager cross-product goes and what moving it costs; we do not show an admission-latency advantage.

1 Introduction

A Pod is the unit of isolation in Kubernetes and also the unit of change. Deployments roll, jobs finish, sandboxes are replaced,

and each event changes who may talk to whom. NetworkPolicy expresses permissions through Pod and namespace labels, ports, and address ranges; the addresses that carry traffic are assigned later and independently. An enforcement engine must join policy with current workload metadata on every node hosting a selected Pod, isolate a new Pod before its containers start, and keep doing both while packets arrive. A service that lives for hours amortizes this setup; a sandbox that lives for seconds does not.

The datapath question is usually framed as lookup speed: chains against sets, iptables against nftables against eBPF. That misses where the cost has moved. nftables applies incremental updates proportional to what changed [26]; Open vSwitch sends misses to userspace and caches results in the kernel [9]; eBPF maps let userspace update keyed state without reloading code [20]. All three make the lookup cheap. None decides how much state must exist before a Pod can communicate, or on how many nodes. An engine that stores one entry per peer identity per direction at each endpoint carries state proportional to identities times local endpoints (Section 4), and each new identity is a write to every endpoint whose policy selects it. Identity sharing among equally labeled Pods limits this; label cardinality and broad selectors restore it. What matters is distinct identities, permission-graph density, and churn, not Pod count.

KNP does not materialize permitted peers. nftables records which local addresses are subject to policy and sends packets with no cached decision to NFQUEUE. A userspace evaluator resolves both addresses to Pod metadata, applies the NetworkPolicy semantics, and returns a verdict. An accept verdict also sets a conntrack label, and a kernel rule then accepts any packet whose connection carries the label in established or related state, so the agent sees a connection roughly once. Denied packets receive no label. Local Pod addresses reach the agent through the runtime’s NRI `RunPodSandbox` hook without waiting for the kubelet to publish them; remote metadata still arrives through API watches.

The research question is whether this trade is favorable for clusters with many identities and high churn, not whether

userspace is faster than the kernel; per packet it is slower. The trade replaces a cross-product of identities and endpoints, maintained eagerly on every node, with per-connection work proportional to communication actually attempted, plus metadata and conntrack that both designs carry. It also introduces a bounded queue whose failure policy defaults to open (Section 3).

Our evidence is archived ClusterLoader2 runs on a 720-worker `n2-standard-4` cluster creating 35,000 Pods at a configured 500 QPS, comparing Cilium v1.20.1 with Kindnet v1.0.1, which bundles KNP. Every sandbox has a `wait-for-gateway` init container that loops until a permitted TCP connection to the gateway Service succeeds under default deny, so a Pod is Running only after the source node’s egress evaluation and the gateway node’s ingress evaluation have both admitted its first connection. The post-scheduling latency we report therefore bounds time to first permitted communication, including propagation of the new Pod’s address to the gateway node. The evidence supports five observations, none of which stands in for another.

1. Cilium’s completed identity sweeps show no rising latency curve: open-gateway P99 after scheduling falls from 5.930 s at 7 identities to 1.990 s at 35,000, hub-and-spoke from 6.370 to 2.033 s, and mean worker CPU stays within 0.51–0.63 cores in every tier of every family.
2. The 35,000-ReplicaSet tiers halve achieved Phase-1 Pod throughput, from roughly 280–330 to 170 Pods/s, and raise cluster-scoped Pod LIST P99 from 14–19 to 28.8 s; the bottleneck there is controller fan-out, not the datapath.
3. Cilium’s bidirectional mesh completes at 7, 700, and 3,500 identities (6.767, 2.934, 2.279 s P99). Its 35,000-identity tier was excluded before execution: two entries per peer identity give 70,000 entries per endpoint against a 65,536-entry policy-map maximum. That is a predicted capacity exclusion, not an observed failure.
4. Two Cilium admission failures are recorded: v1.18.6 stranded 19 of 35,000 Pods at 700 identities, attributed to a policy-cache race fixed in v1.20.0 [1], and a v1.20.1 hub-and-spoke raw-Pod burst aborted with 29,306 Running and 5,694 stranded.
5. KNP’s mesh sweep, matched to Cilium’s generator at the first three tiers, reaches 151.479, 5.331, and 2.791 s P99. Its unmatched 35,000-identity raw-Pod mesh tier completes with every Pod Running at 421.410 s P99 and two startup-SLO failures. At 700 and 3,500 identities KNP created Pods at 103–124 Pods/s where Cilium created 302–328 on the same configured QPS; the artifacts do not record why.

We do not claim an admission-latency advantage over Cilium; the evidence does not show one. KNP is slower at every matched tier, by 0.5–2.4 s at 700 and 3,500 identities and

by $22\times$ at 7. The throughput gap confounds the comparison in KNP’s favor, since a slower generator produces smaller per-node bursts. Each tier has one trial. Every agent-level Prometheus query returned no samples because the deployed Kindnet build exposes no metrics port, so verdict rate, queue occupancy, drops, and agent CPU are absent. The sweep establishes that KNP has no per-endpoint state bound that excludes the dense tier, and that its post-scheduling P99 does not rise from 700 to 3,500 identities.

The paper makes three contributions at three levels of evidence: from source inspection of revision 9984dcd, a description of the enforcement decomposition, its queue-overflow trade-off, and its unknown-peer semantics (Sections 3 and 5); an analytical model of where eager permission state goes under deferred evaluation and of the admission capacity that replaces it (Section 4), whose single service-rate data point is a repository microbenchmark of unrecorded provenance; and an analysis of the archived benchmark collection across both engines, including the newly executed KNP sweep (Section 6), with the missing measurements enumerated in Section 6.9. Section 2 states the problem, Section 7 places the design among reactive caches and factored kernel representations, and Section 8 covers metadata distribution.

2 Background and Problem

Policy semantics. NetworkPolicy isolates ingress and egress independently. Once any policy selects a Pod in a direction, only the union of the applicable policies’ rules is permitted in that direction; rules add permissions and never remove them [19]. A connection between two isolated Pods must pass the source’s egress rules and the destination’s ingress rules, and reply traffic of an allowed connection is implicitly allowed. The semantics are connection-oriented, which makes reusing an admission decision for the rest of a connection natural; the specification prescribes no mechanism.

Lifecycle versus synchronization. Policies and Pods are created asynchronously and the API exposes no barrier that says a policy is enforced everywhere. The expectation is narrower: once an agent has processed a policy, a new Pod it selects must be isolated before its containers start; allow rules may converge later. A system can therefore be safe while delaying legitimate traffic, and a fast transition to Running proves neither safety nor availability. Here a connection gate ties the two together (Section 6).

Four quantities. “Identity scale” conflates four things: live endpoints; distinct policy-relevant label sets; allocated numeric identities, which can outlive the Pods that created them; and materialized datapath entries, which depend on the representation. Several Pods share one identity, one identity contributes an entry to every endpoint that selects it, and permitted

pairs are unrelated to active connections. Section 6 says which quantity each experiment varies.

Where enforcement work is placed. An ordered chain costs work per rule visited, but ipset and nftables sets are indexed and hold throughput across the set sizes Sutter tested [12], and kube-proxy’s nftables backend makes update cost proportional to what changed [26]. NVP and Open vSwitch translate logical networks into per-host actions and cache flow decisions in the kernel with misses handled in userspace [4, 9]; OVN’s stateful ACLs use `allow-related` backed by conntrack [23]. These are the closest precedents for our design, and none is stateless. eBPF maps let userspace mutate keyed state without replacing the program [20]; Cilium keys policy on label-derived numeric identities [15], and the size of that identity space and the capacity of a per-endpoint policy map are different limits. The per-peer cross-product is not the only kernel-resident representation either: OVS conjunctive match installs an $n \times m$ rule as $n + m$ flows [22], Antrea compiles NetworkPolicy onto it and marks established connections in conntrack [13], and Calico factors selectors into ipsets referenced by a bounded number of rules [24].

Problem. Can an engine keep the kernel forwarding path for the bulk of traffic while avoiding eager expansion of every permitted peer at every endpoint? If so, where does the displaced work go, how does the admission point obtain metadata fresh enough to be useful, and what happens to cached decisions when policy changes? Packet throughput answers none of these.

3 Design

3.1 Connection Admission and Cached Decisions

KNP splits enforcement into steering, done by nftables, and evaluation, done by a userspace agent. The agent watches NetworkPolicies, Namespaces, and Pods through the API and learns local Pod addresses from the runtime. From the Pods it derives the *managed* addresses, local non-host Network Pods selected by some policy, and keeps them in nftables sets. A filter chain on the `postrouting` hook at priority `NATSource - 5`, so that it runs before SNAT and sees IPVS traffic that skips `forward`, accepts packets from local sockets owned by UID 0, accepts packets whose connection carries the accept label *and* is in established or related state, and queues to NFQUEUE any remaining packet whose source or destination is in a managed set. An `input` chain does the same for Pod-to-host traffic, queueing only `ct state new` (Figure 1).

The callback parses the headers, resolves source and destination to Pod metadata, evaluates egress on the source and

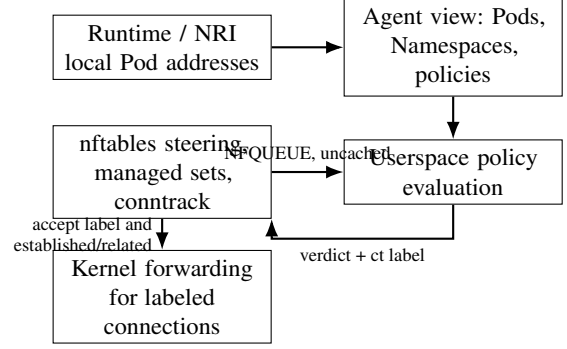


Figure 1: Enforcement decomposition. Protocol and host-traffic exceptions are omitted. API watches feed the agent’s view; NRI supplies local observations only, not remote policy synchronization.

ingress on the destination, and returns a verdict. An accept verdict carries a conntrack label; from then on the bypass rule matches every packet of that connection in the kernel. Denied packets are dropped and receive no label, so the next attempt on that flow is queued again.

Two properties of this cache matter. It is connection state, not a table of permitted peers: cached state is bounded by connections actually made, F_ℓ in Section 4, not by the peers a policy could admit. And “first-packet evaluation” is an approximation: conntrack assigns established state only after seeing the reply direction, so a handshake’s initial packets, retransmissions before the reply, and protocol-specific transitions can each reach userspace. The queued rate exceeds the connection rate and includes every denied attempt.

nftables keeps a role. It holds the steering rules and the managed sets, and a change in which local Pods are selected is a set update; it does not execute label selectors. Avoiding the peer cross-product does not remove local rule maintenance, and the evaluator’s indexes over Pods, Namespaces, and policies cost CPU and memory. Once metadata is present the decision is local; nothing on the packet path allocates a distributed identity. Remote Pod metadata must still reach the node.

3.2 Runtime Metadata and Startup Isolation

If the agent learns its own node’s Pod addresses only from the API, the chain is: the runtime creates the sandbox and the CNI assigns an address; the kubelet writes Pod status; the API server fans it out; the agent’s watch delivers it. Containers may start, and lifecycle hooks may begin waiting on the network, before the last step; issue 85966 records the resulting failures [11]. KNP’s NRI resolver populates the local IP-to-Pod cache in its `RunPodSandbox` callback, from the addresses the runtime reports or by inspecting the sandbox network namespace, so local address discovery no longer

waits for the kubelet-API-server-watch round trip.

Two limits apply. Knowing an address is not enforcing on it: the steering path must cover the Pod before its containers run, which depends on set updates and callback ordering, not on NRI alone. And the hook is local: policies, namespace labels, and remote peers still arrive asynchronously. We claim removal of one asynchronous dependency, not zero startup latency, and no NRI on/off comparison exists to quantify even that.

3.3 Policy Changes and Failure Semantics

Caching makes policy updates part of the correctness argument. The specification leaves the effect of an update on established connections implementation-defined [19]. KNP’s strict mode scans conntrack, reevaluates supported established flows, and clears the accept label where current policy no longer permits the flow, so the next packet is queued and re-decided. Revocation latency is watch delivery plus scan scheduling, scan duration, and packet arrival, and scan cost grows with the flows inspected.

Queue overflow. Any design that decides in userspace behind a bounded kernel queue must choose what the kernel does when the queue is full. NFQUEUE offers two answers, drop or accept unevaluated [21]; KNP exposes the choice as a flag and its queue holds 1,024 packets. This is the counterpart of the eager design’s overflow choice, where Cilium can lock down an endpoint whose policy map is full [14]: in both places the operator trades availability against isolation at a capacity boundary. Denied packets are never labeled, so denied traffic is part of the queue’s load rather than of the cached path. The archived runs did not measure queue occupancy or drops and did not approach overload, so they characterize neither mode; the single-node experiment in Section 6 is where that boundary would be measured.

Unknown peers fail closed. When the evaluator cannot resolve a remote address to a Pod, the peer is `nil`, and a `nil` peer matches only `ipBlock` rules. Under default deny the packet is dropped. That is the correct isolation behavior and an availability cost: a permitted connection from a freshly created remote Pod is denied until the destination node has that Pod’s metadata. NRI shortens this only for local Pods; for remote peers the node depends on watch delivery or on the distribution tier of Section 8.1. The benchmark exercises exactly this path: a sandbox’s first connection is admitted at the gateway node only after that node has learned the sandbox’s address, so the post-scheduling latencies in Section 6 contain this convergence.

Safety property and trust boundary. The property we want is conditional: for a policy an agent has already pro-

cessed, a newly selected Pod must not send or receive forbidden traffic before enforcement applies; a permitted connection may be delayed by missing metadata, and that delay is not a violation. The archived runs do not establish it. They exercise the permitted sandbox-to-gateway path on every Pod and the forbidden path once per cluster, by hand, unarchived; missing metadata, label changes, and address reuse are untested. The trust boundary includes the node kernel, the runtime, the agent, and the CNI’s binding of addresses to Pods. A workload that can spoof a peer’s address defeats the evaluator; `hostNetwork`, privileged Pods, and address translation need explicit scope that NFQUEUE and NRI do not provide.

4 Where the Scaling Cost Moves

Let I be the distinct peer identities a policy selects, P_ℓ the local endpoints it applies to, d the directions materialized, and F_ℓ the active connections tracked on the node. An engine that stores one entry per peer identity per direction at each endpoint carries

$$E_{\text{endpoint}} \approx dI, \quad E_{\text{node}} \approx dIP_\ell. \quad (1)$$

This describes one representation, not a lower bound: sharing, wildcards, aggregation, or a factored match (Section 7) change it, other rules add to it, and entries are not bytes. Under the same assumptions a new identity selected by every local endpoint costs about dP_ℓ writes, and a rate λ_I of new identities costs $dP_\ell\lambda_I$ writes per node before dissemination and selector recomputation. Replacing Pods that reuse a label set leaves this nearly unchanged; fresh-identity churn is a different workload from Pod churn. The benchmark tiers vary I ; whether their churn rounds create fresh identities is not recorded.

Deferred evaluation removes the dIP_ℓ term and keeps everything else:

$$M_{\text{deferred}} = M_{\text{metadata}} + M_{\text{steering}} + M_{\text{conntrack}}(F_\ell). \quad (2)$$

This is an accounting, not a proof of lower total memory: both designs carry conntrack, and the userspace Pod and policy indexes can dominate. What is removed is materialized permission state; what replaces it is work at admission time.

That work has a capacity. Let λ_q be the rate of packets reaching the queue, including denied attempts and pre-establishment retransmissions, and μ the rate at which the agent returns verdicts. The first-order condition for the queue not to grow is

$$\rho = \lambda_q / \mu < 1, \quad (3)$$

and because the queue is 1,024 packets deep, exceeding it is the overflow boundary of Section 3, not only latency; bursts and CPU contention can reach it below $\rho = 1$. Our only measurement of μ is the repository’s three-node ApacheBench run [18]: 10,000 non-keepalive requests at concurrency 1,000 completed at 2,317 requests/s, P50 5 ms, P99

3,080 ms dominated by connect time, with the kernel logging `nf_conntrack: table full` at the default 262,144 entries; its charts show queued-packet peaks near 8,000/s per node and callback durations of 80–120 μ s median and 170–630 μ s P99. It is a 2024 measurement with no hardware, kernel, or agent version recorded, and conntrack exhaustion confounds the tail; we treat it as an order of magnitude for one node, not as μ . The 720-node runs measured neither λ_q nor μ .

The favorable regime is therefore precise: a permission graph that is large or changing fast relative to the connections actually attempted, with p well below one on every node. A dense permission graph with sparse traffic fits. A dense connection graph, frequent revocation, sustained denied traffic, or many short connections does not; there the eager engine pays once per identity and the deferred engine pays per connection.

5 Implementation

The inspected code is the Go kube-network-policies repository at revision 9984dcd. The dataplane controller programs nftables over netlink and opens NFQUEUE with a maximum queue length of 1,024 packets, a copy range of 128 bytes, GSO enabled, and a 100 ms write timeout. These are configuration constants, not measured service rates.

The callback times each packet, records the duration in a histogram labeled by protocol and family, and counts verdicts by protocol, family, and outcome; queue occupancy and drops are exported as well. Callback time starts at delivery to userspace, so it excludes queueing before delivery and the verdict’s return path; it is not connection-setup latency as an application sees it. None of these series exist for the archived runs: the deployed Kindnet build exposes no metrics port, and every agent query in the harness returned no samples. DNS, ICMPv6 neighbor discovery, and loopback exceptions are additional rules omitted from Figure 1.

The repository builds several agent flavors. The standard flavor queues only traffic to or from managed addresses. The IPTracer flavor, at the same revision, returns *divert-all* from ManagedIPs: it watches NetworkPolicy objects but has no Pod informer from which to enumerate selected local Pods, so it queues every forwarded packet not already accepted by conntrack, and the queue-load model applies to a larger traffic fraction. The NRI resolver is an alternate source of local Pod records, added and removed by runtime callbacks; namespace metadata still comes from the API view.

One naming distinction is essential for reading the evaluation. KNP names the engine inspected here; the benchmark labels complete network stacks *Kindnet* and *Cilium*. The KNP cluster was created with kops’s `kindnet: {}` networking, patched to `kindnet:v1.0.1` with an NRI socket mount by `scripts/create-netpol.sh`; the repository’s standalone install manifest pins `kube-network-policies:v1.1.0`. Which policy binary and flags were effective in each Kindnet run is not archived, and neither image is matched to revision

9984dcd. The latency differences in Section 6 are differences between stacks, not attributable solely to the code described here.

6 Evaluation

6.1 Methodology and Provenance

Every number in this section is read back from archived ClusterLoader2 output; nothing was re-measured for this draft, and the extractor (`submission/analyze.mjs`) and its inputs are listed in Appendix A. The testbed was built with kops on GCE: 720 `n2-standard-4` workers, one `c4-standard-96` control-plane node, Kubernetes 1.36.2, containerd 2.2.4 with runc 1.3.5. Two clusters were created from the same specification and differ in CNI, kube-proxy configuration, and address ranges: `agentic-cilium.k8s.local` runs Cilium v1.20.1, and `agentic-netpol.k8s.local` runs Kindnet v1.0.1 with its bundled KNP agent and an NRI socket mount. Whether that bundled binary matches the revision inspected in Section 5 (9984dcd) is not established: image digests and effective agent flags were never written into `cl2-metadata.json`, and the standalone install manifest in the repository, which pins `kube-network-policies:v1.1.0`, is not what the cluster ran. Every archived run targets 35,000 concurrent sandbox Pods at a configured 500 object operations per second with `default-deny-policy.yaml` applied (`ApplyDefaultDenyPolicy: true` in every saved configuration).

The extraction covers 66 run directories and recovers 59 startup reports; a missing report is treated as missing, never as a zero. Commits 0442dce and f998224 add the raw-Pod abort record and the Cilium mesh-tier exclusion note, documented outcomes with no latency file; 5b152f2 adds the KNP mesh sweep.

Metrics. ClusterLoader2 writes `create_to_schedule`, `schedule_to_run`, and `total pod_startup percentiles` in milliseconds; we report seconds to three decimals and never add or average percentiles across components or runs. The middle metric is where this harness differs from a plain container-startup benchmark. Every sandbox template carries a `wait-for-gateway` init container that loops `nc -z -w 1` against the gateway Service until a TCP connection succeeds. Under the deployed policies a sandbox may send only to `group: gateway` on TCP 80 and the gateway may receive only from `group: sandbox`, so a sandbox reaches `Running` only after one *permitted* connection has been admitted by the egress evaluation on its own node and by the ingress evaluation on the gateway’s node. `schedule_to_run` is therefore an upper bound on time to first permitted communication for the sandbox-to-gateway path, and it includes propagation of the new Pod’s address to the gateway node, the metadata-convergence cost Section 3 is about. Four caveats bound that

reading: the gate covers one path, not sandbox-to-sandbox; it exercises no forbidden path; a queue configured to accept on overflow would also let the probe through, though no run approached overflow; and although the init container logs its wait as Latency: N ms, those per-Pod lines were not collected, so within a sample the gate cannot be separated from image pull and container start.

Negative enforcement was checked once per cluster, not once per run. `scripts/verify-netpol.sh` creates a client and a server, confirms they can talk, applies a default-deny ingress policy and confirms the request is blocked, then applies an explicit allow and confirms it succeeds. It was run on each idle cluster after creation; no run script invokes it and its output is not archived. The benchmark thus establishes positive enforcement per Pod at scale, and negative enforcement only as a quiescent smoke test.

Trials. Saved `generatedConfig_*.yaml` files take precedence over the current generator. The September sweeps vary Pods per ReplicaSet together with intended identity cardinality, use the default scheduler, and replace 5,000 Pods in each of five churn rounds. Five rounds inside one run are not five trials; every cell in Tables 1–3 is a single run. Intended identities are the generator’s label cardinality; allocated Cilium identities were not dumped. A configured QPS is an object-operation rate, and one ReplicaSet operation can create 5,000 Pods, so achieved Pods per second is reported separately in Table 2.

6.2 Workload Admission and Identity Turnover

Table 1 collects the completed September tiers on both clusters: Cilium under an open gateway, under hub-and-spoke, and under the bidirectional mesh policy, and KNP under the same bidirectional mesh. Figure 2 plots the same values on a logarithmic axis so that the 151-s and 421-s KNP points and the 2-s Cilium points fit on one panel.

Within the ReplicaSet-paced tiers from 700 identities upward, neither engine shows a rising post-scheduling tail. Cilium’s open-gateway P99 falls from 5.930 s at 7 identities to 1.990 s at 35,000 (hub-and-spoke: 6.370 s to 2.033 s). In the mesh, KNP goes from 5.331 s P99 at 700 identities to 2.791 s at 3,500 (medians 3.273 s and 2.234 s), against Cilium’s 2.934 s and 2.279 s on the identical generator: within 2.4 s and 0.5 s. Cilium’s mesh medians are 1.412, 1.424, and 1.413 s across its three tiers, so Cilium’s typical Pod is faster than KNP’s at every matched tier, and the tails converge only at 3,500 identities.

The two remaining KNP tiers are not flat, and they are why the figure needs a log axis. At 7 identities with 5,000 Pods per ReplicaSet, KNP recorded 38.946 s P50 and 151.479 s P99 after scheduling where Cilium recorded 1.412 s and 6.767 s under the same configuration: a $22\times$ P99 gap with zero JUnit

Intended identities	Pods/RS	Cilium P99 (s)			KNP Mesh P99
		GW	Spoke	Mesh	
7	5,000	5.930	6.370	6.767	151.479
700	50	5.416	4.759	2.934	5.331
3,500	10	4.685	4.899	2.279	2.791
35,000	1	1.990	2.033	Excl.	—
35,000	Raw	—	Abort [†]	—	421.410*

Table 1: Per-run archived schedule-to-run P99. All ReplicaSet-paced suites have zero JUnit failures. [†]Cilium raw-Pod hub-and-spoke run aborted at 29,306 Running / 5,694 stranded; no latency report. *KNP raw-Pod mesh run: 35,000 Running, 0 stranded, two JUnit failures against the 5-minute P90 startup SLO (observed 357.7 s). The two raw-Pod runs use different policies and are not a matched pair.

failures on either side, and nothing in the artifacts explains it. It is consistent with the July Kindnet baseline (57.068 s P99) and tuned bundle (9.718 s after scheduling, 95.790 s total) at the same 5,000-Pods-per-ReplicaSet shape, so the degradation tracks large-ReplicaSet bursts rather than identity count.

The 35,000-identity KNP tier was run as 35,000 raw Pods under the mesh policy and has no Cilium counterpart: the Cilium producer excluded the mesh tier analytically (Section 6.5), and Cilium’s only raw-Pod run used the hub-and-spoke policy and aborted (Section 6.6). KNP completed it with all 35,000 Pods Running and none stranded, at 120.003 s P50 / 421.410 s P99 after scheduling and 139.557 s / 499.865 s end to end; the harness recorded two JUnit failures against its 300-s P90 startup SLO (observed 357.7 s). `create_to_schedule` alone reached 23.987 s P50 and 106.691 s P99, so the scheduler was more than a minute behind for the tail before any node did work. Candidate causes for the post-scheduling tail are serialized sandbox creation in kubelet (about 49 Pods land on each node), convergence of 35,000 new Pod addresses to the gateway nodes before their init containers can be admitted, and API-server watch latency; the kubelet CNI-latency query and all six KNP agent queries returned no samples, so the artifacts do not separate them.

These results rule out the simple story that identity cardinality by itself makes Cilium’s admission collapse. They do not prove identity-independent scaling for either engine, because the generator changes controller fan-out and burst size with every tier: Table 2 shows Cilium’s achieved Phase-1 throughput at 277–328 Pods/s (median of per-second samples) in every tier with 10 or more Pods per ReplicaSet and 169–170 Pods/s in the two 35,000-ReplicaSet tiers, with cluster-scoped Pod LIST P99 rising from 14–19 s to 28.8 s. The lowest Cilium tails coincide with a controller-limited creation stream that halves the burst each node absorbs.

The mesh tiers are the only place the two engines meet on the same generator and policy, and two confounds still

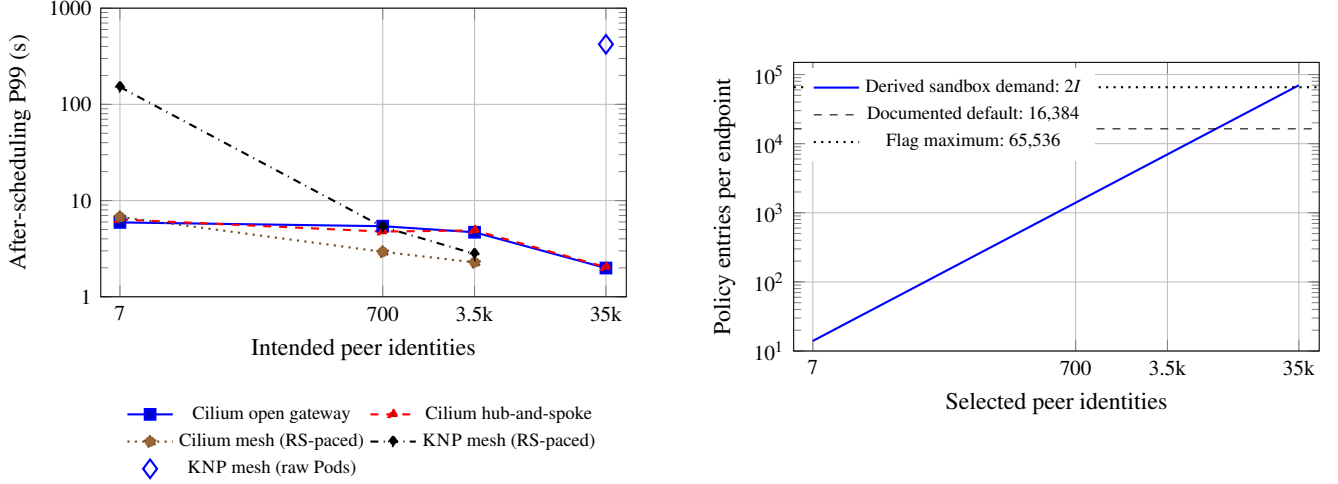


Figure 2: Two meanings of scaling. Left: measured per-run after-scheduling P99 on a logarithmic axis so that every executed tier is visible; connecting lines are visual guides, each point is one run, and the raw-Pod point has no matched Cilium run. Right: conditional permission-state model for eager per-endpoint compilation (Equation 1 with $d = 2$). The runner’s header records `-bpf-policy-map-max` as 65,536 on the Cilium cluster, which admits the unidirectional 35k tiers and excludes the bidirectional mesh at 35k ($2I = 70,000 > 65,536$); the effective value was not archived.

prevent reading them as an engine comparison. First, the KNP runs created Pods at 124 and 103 Pods/s median against Cilium’s 302 and 328 on the same configured 500 QPS and identical control-plane settings, so each KNP node absorbed Pods at well under half the rate; by the argument just made that alone lowers the tail, and the cause of the slower stream on the Kindnet cluster is not recorded. Second, each cell is one trial. What the matched tiers do support is narrower: under a generator Cilium handles at 2.3–2.9 s P99, KNP handles 700 and 3,500 identities at 2.8–5.3 s P99 with the gateway-admission gate included, and its worker CPU does not rise with identity count. Its end-to-end P99 at those tiers, 5.553 s and 2.918 s, is far below Cilium’s 28.739 s and 21.730 s only because KNP’s `create_to_schedule` P99 was 0.418 s and 0.231 s; that component ends before any node or policy agent is involved.

6.3 Resource and Control-Plane Measurements

Table 2 lists what the harness recovered about the creation stream and the workers. Phase-1 throughput is the median and P90 of per-second scheduled-Pod samples during the initial fill; CPU is the September `instantaneous_node_cpu_cores` query aggregated across all 720 4-vCPU workers over the run.

Worker CPU does not track intended identity count in either family. Cilium’s mean stays within 0.51–0.63 cores and its P99 within 0.67–0.86 across all eleven tiers and scenarios; KNP mesh stays within 0.45–0.66 mean and 0.56–0.86 P99. The two KNP burst tiers show single-node instantaneous

maxima of 2.19 cores (tier 1) and 2.55 cores (tier 4), but the maximum of scrape-interval samples is not a peak. All of this is a whole-node aggregate that includes kubelet, containerd, and the sandbox workload itself; it cannot say what share the policy agent took. It does exclude one thing: there is no CPU explosion on the workers at 35,000 identities. If Cilium pays for identity cardinality in these configurations, it pays in the operator, the API server, or in map capacity, not in visible worker CPU. Cluster-scoped Pod LIST P99 on the Kindnet cluster is 19.05, 8.24, 14.05, and 1.46 s across the four mesh tiers, the last belonging to the raw-Pod tier whose scheduler was 106 s behind, so API latency and scheduling backlog are different symptoms. The `kindnetd` CPU term in the saved query has no samples in any run, and every Cilium pprof capture requested through the harness failed with connection refused; the λ_q , μ , and M_{deferred} terms of Section 4 are unmeasured here.

6.4 Version Dependence

Version matters as much as cardinality. An archived Cilium v1.18.6 attempt at the 700-identity gateway tier, run three hours before the v1.20.1 sweep on the same day, retains only its generated configuration and `cl2-metadata.json`; the harness never produced a startup report. The repository’s narrative records 34,981 Pods Running and 19 stranded in `Init:0/1` under default deny, and attributes the outcome to a policy-cache race between identity allocation and endpoint regeneration tracked as Cilium issue 7515 and fixed in v1.20.0 [1]. We have not reproduced that diagnosis, because no logs from the attempt were kept. The narrower

Scenario	Identities	Pods/s		Node CPU (cores)	
		P50 / P90	mean	P99	
Gateway	7	284 / 377	0.52	0.71	
	700	317 / 364	0.51	0.69	
	3,500	316 / 372	0.54	0.69	
	35,000	170 / 182	0.56	0.81	
Spoke	7	0 [†] / 327	0.51	0.69	
	700	323 / 398	0.61	0.86	
	3,500	321 / 373	0.54	0.67	
	35,000	169 / 181	0.55	0.78	
Cilium Mesh	7	277 / 376	0.54	0.72	
	700	302 / 351	0.54	0.67	
	3,500	328 / 385	0.63	0.77	
KNP Mesh	7	0 [†] / 310	0.52	0.66	
	700	124 / 134	0.48	0.61	
	3,500	103 / 106	0.45	0.56	
	35,000 (Raw)	0 [†] / 97 [‡]	0.66	0.86	
Kindnet QPS 500	~7	233 / 262	0.40	0.46	
Kindnet tuned	~7	190 / 225	0.63	0.79	

Table 2: Recovered Phase-1 scheduling throughput and worker-node CPU for the runs in Tables 1 and 3. CPU is the harness’s instantaneous node query on 4-vCPU workers, aggregated across nodes over the run; the Kindnet QPS-500 row uses an earlier rate-based query and is not directly comparable. [†]More than half of the per-second samples were zero. [‡]Peak 841 Pods/s. No memory series exists anywhere, and the six KNP agent queries added for the mesh sweep (CPU, memory, packet processing time, verdict rate, queue drops, user drops) returned `dataItems: null` in every tier because the deployed Kindnet build exposes no metrics port.

point stands on the artifacts alone: the same generator at the same tier produced an admission failure on one release and a clean 5.416-s P99 on the next. `Init:0/1` is the state a sandbox stays in while `wait-for-gateway` is still looping, so if the narrative’s counts are right, those 19 Pods are the one recorded instance of the enforcement gate holding Pods back. Identity-scaling claims about eager-materialization systems are release-specific, and a comparison has to pin and report the version it ran.

6.5 Capacity Exclusion Is Not a Latency Sample

The mesh runner never attempted the 35,000-identity tier on Cilium. Its `MESH_SWEEPS` list is `(5000 50 10)`, omitting one Pod per ReplicaSet, and it carries a guard that skips a listed tier when the estimated per-endpoint entry demand reaches the configured map limit. The model behind the guard assigns one entry per sandbox identity per direction at every endpoint; Equation 1 with $d = 2$ gives about 70,000 sandbox

entries per endpoint at 35,000 identities, before gateway, DNS, or reserved-identity entries, which exceeds the 65,536-entry value the runner documents. We call this an *analytical exclusion*: design evidence that a system can have low latency throughout its feasible region and still hit a state-capacity boundary. It is not evidence that an executed 35,000-identity mesh on Cilium experienced insertion failures, packet loss, or any particular latency, and it has no P99 to plot.

The effective `-bpf-policy-map-max` on the Cilium cluster is not archived. In v1.20.1 source the flag defaults to 16,384 and is clamped to at most 65,536 [14]; the runner reads the value from the `cilium-config` ConfigMap at run time, falls back to 16,384 if it is unset, and its header comment records the tested value as 65,536. The completed runs corroborate the higher value independently. The gateway policy admits ingress from every sandbox identity, so at 35,000 identities each gateway endpoint needs about 35,000 entries; behind a 16,384-entry map, Pods past that count could not pass their `wait-for-gateway` gate and would have stayed in `Init:0/1`. All 35,000 reached `Running` in both unidirectional 35,000-identity tiers with zero JUnit failures, so the effective limit was at least 35,000 unless v1.20.1’s identity aggregation applied, and the source restricts aggregation to cluster-, world-, and remote-node entity buckets rather than label selectors. One `cilium-dbg bpf policy get dump` on a gateway node would have settled the value; none was taken. The agent also offers an `endpoint-lockdown-on-overflow` option [14], so an executed overflow would surface as denied traffic rather than as a startup latency, one more reason the unrun tier has no P99.

Under the same model, 3,500 selected identities and 50 local endpoints imply roughly 350,000 sandbox policy entries per node: derived demand, not a map dump. The contrast with KNP is structural rather than measured. Its design has no per-endpoint permission cross-product, so no capacity guard excluded its 35,000-identity mesh tier, and that tier completed with every Pod `Running`. Its limits sit in metadata storage, conntrack sizing, and userspace verdict rate, and the 421-s tail of that same tier is the reminder that removing the map bound does not by itself make the tier cheap.

6.6 An Aborted Direct-Burst Run

The raw-Pod microsegmentation record on the Cilium cluster documents a different outcome. A run that created Pods directly, without ReplicaSets, at the configured 500-QPS tier under the hub-and-spoke policy aborted with 29,306 `Running` and 5,694 permanently stranded out of 35,000. The record attributes the abort to two mechanisms: concurrent CNI ADD calls hitting the local Cilium REST API’s HTTP 429 limit, and cumulative identity allocation across creation and churn exhausting the 65,280 allocatable cluster-local identities, with the allocator reporting “no more available IDs in configured space.” The harness produced no final latency report, so the

run appears in Table 1 as a completion count and nothing else.

A missing percentile file is not a reason to drop a failed run, nor to promote the documented diagnosis to a reconstructed timeline. Allocator history, identity garbage-collection behavior, effective limiter settings, and the agent logs would be needed to separate the two causes and order them; none were retained. The identity allocator is cluster-level state, distinct from the per-endpoint policy-map capacity of Section 6.5; 65,280 and 65,536 are different limits on different structures. And a direct Pod burst is a different workload from ReplicaSet-mediated creation, which is why this run and the KNP raw-Pod mesh tier are separated in Table 1 rather than compared. The abort supports an admission-failure case study, not a claim that every 35,000-identity workload fails on Cilium.

6.7 Connection Cost and Correctness

The collection measures workload admission and the control-plane activity around it, not the core packet-admission tradeoff of Section 3: NFQUEUE verdict capacity, established-flow throughput parity, and isolation through startup and failure are all unmeasured. Because the `wait-for-gateway` gate folds one admitted connection into every `schedule_to_run` sample, the sweeps do bound first-permitted-connection latency on one path, but they cannot isolate that component from image pull and container start, a configured mesh permission does not generate all-to-all traffic, and reaching `Running` tests no forbidden peer relationship. The only negative check is the one-shot smoke test of Section 6.1.

The only connection-level data point anywhere is the implementation repository’s own three-node microbenchmark summarized in Section 4: 10,000 non-keepalive HTTP requests at concurrency 1,000 completed at 2,317 requests/s through the queue with a 5-ms median and a 3,080-ms P99 while the kernel logged `nf_conntrack: table full` at 262,144 entries, with per-node queued-packet peaks near 8,000 packets/s and callback medians of 80–120 μ s [18]. It dates from April 2024 and records no hardware, kernel, or agent version, so we treat it as indicative. Read against the sweeps, it says that a node absorbing about 50 sandboxes while the cluster creates 103–328 Pods/s is nowhere near the queue’s connection limit, and that on that node the limit was reached first by conntrack capacity, not by the userspace evaluator. It does not say what happens under sustained denied traffic, whether the 3-s P99 is queueing or conntrack, or how either changes with policy count. Nothing in the cluster artifacts exercises the IPTracker flavor of Section 8.1, whose divert-all steering would give the same workload a different λ_q . The discriminating experiments, and the order in which they should be run, are in Section 8; the first of them is a single-node connection microbenchmark with a denied-traffic mix, run into overload in both overflow modes, to locate the capacity boundary of Section 3.

QPS tier	After scheduling P99		Total P99	
	Cilium	Kindnet	Cilium	Kindnet
50*	4.065	4.833	4.089	4.855
100*	1.788	7.143	1.814	7.378
200	1.845	59.347	2.285	59.485
500	5.482	57.068	11.160	57.629

Table 3: Archived primary QPS runs, July 2026; all values in seconds. Asterisks mark tiers whose only JUnit failures are final-teardown deletion timeouts in both configurations; startup measurements completed. These are complete-stack observations on two clusters, not an isolated NFQUEUE-versus-eBPF comparison.

6.8 Control-Plane Sensitivity

The July QPS sweeps are why startup has to be decomposed at all. Table 3 reports the primary saved Cilium and Kindnet baseline runs. At QPS 500 the after-scheduling P99s are 5.482 s and 57.068 s (medians 1.490 s and 7.922 s), so the archived baseline does not establish any startup advantage for the NFQUEUE-based configuration; at the burst shape the July runs used it shows the opposite. At QPS 50 and 100 both families record two JUnit failures. All four are the same measurement, `WaitForOldPodsDeleted-Final`, timing out while 5,024 to 19,999 churned sandbox Pods were still terminating. No startup-phase measurement failed, so the startup percentiles in those rows are complete, but the runs did not finish teardown within the harness deadline, and Pod deletion latency at those tiers is unmeasured.

The Kindnet cluster was then tuned in five steps that changed the agent version, the NRI socket mount, and API Priority and Fairness (APF) settings together. The final bundle, labeled v1.0.1 with NRI and an APF exemption, records an after-scheduling P50 of 2.345 s and P99 of 9.718 s. Its total P99 is nevertheless 95.790 s, with a create-to-schedule P99 of 94.555 s: the tail moved from the node to the scheduler. That is direct evidence against reading a reduced post-scheduling tail as an end-to-end speedup, and because the variables changed together the sequence is a tuning chronology, not an NRI ablation. A separate Cilium QPS-500 repetition records 4.257 s after scheduling and 12.404 s total; it is kept as a second trial beside the 5.482 s and 11.160 s primary, not averaged with it. The recovered throughput rows add one more caution: the verified Cilium QPS-500 run achieved 180/412 Pods/s P50/P90 while the Kindnet baseline achieved 233/262 and the tuned bundle 190/225, so at this tier the Kindnet cluster’s median creation rate was higher than the Cilium cluster’s when its tail was worst, the reverse of the mesh-tier confound.

APF divides API-server concurrency among request classes and provides queueing and isolation between them [16]. Exempting a class or raising its share moves contention to other classes; it does not create capacity. The wider limitations apply to everything above: most configurations have one trial,

Missing evidence	Claim at stake	Cheapest producer
Agent CPU, memory, verdict rate, latency, drops (all six queries empty)	where displaced work lands; λ_q, μ , overflow margin	rebuild Kindnet with the metrics server enabled; rerun one tier
Cause of the KNP 151 s and 421 s tails	burst behavior of the KNP node path	collect init-container Latency: logs and kubelet PLEG/CNI metrics on a sample of nodes
Why KNP runs created Pods at 103–124 Pods/s versus Cilium’s 302–328	validity of the matched-tier comparison	compare controller-manager and API-server metrics of the two clusters
A Cilium raw-Pod mesh run or a KNP 1-Pod-per-ReplicaSet 35k run	a matched pair at 35,000 identities	one additional run on either cluster
Repeat trials	any statistical statement	rerun tiers 2–3 on both clusters
Policy-map occupancy dump and the effective map-size flag	capacity model of Fig. 2	cilium-dbg bpf policy get and the ConfigMap on one node
Denied traffic under load	isolation, deny amplification	single-node connection microbenchmark with deny mix, both overflow modes
NRI on/off with identical binaries	startup-metadata benefit	rerun baseline QPS 500 and tuning bundle 1 with one pinned image
Binary digests and flags per run; logs for both aborts	attribution and causal diagnosis	record in c12-metadata; retain logs on abort

Table 4: Evidence still absent from the benchmark and implementation repositories after the KNP mesh sweep.

summary percentiles hide time variation and preclude per-Pod trajectories, failed JUnit elements are not failed Pods, and instantaneous CPU maxima at the scrape interval are not peaks.

6.9 What the Artifacts Still Do Not Contain

The KNP mesh sweep closes the largest gap of the earlier draft: a KNP run above seven identities now exists at 700, 3,500, and 35,000. It does not close the comparison. The six KNP agent queries and the kubelet CNI-latency query added for that sweep returned no samples in all four tiers, and the repository’s own probe script concludes that the deployed Kindnet build has the agent metrics endpoint disabled. Table 4 lists what remains: the missing evidence, the claim it would support or refute, and the cheapest experiment that would produce it.

Two absences are cheap to fix: the per-Pod Latency: N

ms lines that every init container already prints would, if scraped, split the gate from container start in every sample, and archiving `scripts/verify-netpol.sh` output per run instead of once per cluster costs seconds. Until both are done, the enforcement claim rests on a per-Pod positive gate and a single unarchived negative check.

7 Related Work

The right ancestor of KNP is not another consumer of a kernel packet API; it is reactive flow processing. Open vSwitch classifies in userspace and caches the result in a kernel flow table, and Pfaff et al. are explicit about what that split costs when the cache misses [9]. NVP realized distributed logical networks on that substrate [4], and OVN’s stateful ACLs use allow-related backed by connection tracking, so a decision taken on a connection’s first packet is inherited by the rest of it [23]. Userspace miss handling, cached verdicts, and conntrack-backed stateful filtering are all prior art, and this paper claims none of them. What it claims is narrower: evaluation of Kubernetes NetworkPolicy semantics in userspace at connection admission, without materializing the permitted-peer set at each endpoint in advance, with local Pod metadata taken from the container runtime rather than the API server. The novelty, such as it is, lies in the placement and the object cached, not in the mechanism.

Andromeda and ONCache are the nearest treatments of what to cache and when to invalidate it. Andromeda separates its flow-processing paths so that provisioning, isolation, performance, and feature velocity can be traded independently [2]; ONCache finds work repeated across the layers of a container overlay, filtering included, and caches the invariant part in eBPF [5]. KNP caches one object, the per-connection verdict, and its invalidation story is the strict-mode conntrack scan of Section 3. KUBEDIRECT names API-mediated controller communication as a serverless scaling bottleneck and builds state management for a more direct path [10]; the same observation motivates KNP’s use of NRI, which removes one asynchronous round trip for one class of local facts and provides none of KUBEDIRECT’s controller coordination. Role-based microsegmentation infers roles and policies from observed traffic [6]; it produces the policy an engine then has to enforce and says nothing about the cost of enforcing it.

The point most likely to be lost in Section 4 is that eager peer expansion is one kernel-resident representation among several, and not the one the other kernel-based enforcers cited here use. Open vSwitch’s conjunctive match installs a rule with n sources and m destinations as $n + m$ flows sharing a `conj_id` rather than $n \times m$ flows [22], and Antrea compiles NetworkPolicy exactly this way, marking established connections in conntrack so that only new flows walk the policy tables [13]: the same evaluate-once-per-connection property this paper attributes to NFQUEUE plus conntrack, obtained without leaving the kernel. Calico’s Felix compiles selectors

into ipsets referenced by a bounded number of iptables or nftables rules, so per-endpoint state grows with rules rather than peers [24]. nftables sets offer the same factoring to anyone; Sutter’s measurements [12] and kube-proxy’s incremental nftables updates [26] exist because lookup cost and update cost are separate questions. eBPF maps are likewise keyed state that userspace updates incrementally [20]; the $2I$ entries per endpoint of Equation 1 describe how Cilium chooses to key its per-endpoint policy map by security identity [15] under a `-bpf-policy-map-max` that defaults to 16,384, tops out at 65,536, and can optionally lock the endpoint down on overflow [14]. That is one representation under one configuration. A capacity comparison that means to say something about kernel enforcement as a class needs a factored-kernel baseline, and this paper’s does not have one.

Finally, a distribution tier between the datastore and the node agents is established practice: Calico’s Typha holds datastore state, deduplicates events, and fans updates out so that each agent does not carry its own load against the datastore [25]. IPTracker has to be judged on its representation, caching, and recovery behavior; the tier itself is not a contribution.

8 Discussion and Future Work

8.1 Compact Metadata Distribution with IPTracker

Declining to run a distributed identity allocator does not make distributed metadata go away. If N agents each watch every Pod and Pods change at rate U , the API server delivers on the order of NU events; filtering, the watch cache, and event size move the constant of that first-order count, not the product. IPTracker, introduced in PR 190 [7], moves the full Pod watch to R distributors and exposes compact IP-to-Pod records to the agents, so full Pod subscriptions against the API server fall from N to R . Other subscriptions remain, and the fan-out moves rather than disappears: the inspected client watches the entire compact-record prefix, a smaller object delivered to the same number of subscribers rather than a selective subscription to the peers a node will actually contact.

Records are Protocol Buffers, optionally persisted in bbolt with an in-memory LRU in front [17]. Kubernetes already speaks protobuf; the distinction is the reduced schema and the topology. The details are less flattering. The LRU admits updates as well as lookups, so churn among peers a node never contacts can evict the ones it contacts constantly; store access is serialized under a mutex, coupling write bursts to lookup latency; the full sync clears local state and reinserts the prefix, which is neither an atomic snapshot replacement nor a constant-memory bootstrap; and a bounded object cache bounds neither process nor machine memory once bbolt’s mapping, page cache, and full-sync buffers are counted. Issue

114659 documents why informer memory is worth reducing [3]; it does not measure this implementation.

The only quantitative data are single-client integration logs posted with PR 218 [8]: 10,000 records gave 643 ms average propagation, 35 MB of server Go heap, and a 1 MB client store; 40,000 records at 30,000 writes/s gave 6.6 s and 174 MB, with the etcd maintainer in the thread reporting watch stalls above that size; 10 million records throttled to 1,024 writes/s finished in 9,828 s at 43 ms average propagation, 24.7 GiB of server heap, and a 1.25 GiB client store. One process, one machine, no packet path, an LRU sized to hold every record: these bound neither agent memory under eviction nor fan-out to hundreds of clients. They do show that the distribution tier, not the client, holds the bulk of the state, and that in the tested range propagation tracks write rate rather than record count. Before persisted records serve as authorization metadata the design owes answers on revision recovery after compaction, reflector failover, disk loss, and above all IP reuse: a stale record for a reused address is a policy bypass, not an availability delay. The IPTracker plugin flavor also diverts all forwarded traffic to the queue (ManagedIPs reports every address as managed), so its λ_q differs from the one in Section 4. IPTracker is implemented future work; no result in Section 6 used it or supports it.

8.2 Testing the Architectural Hypothesis

Section 6.9 lists what is missing; this is the order in which we would collect it, by how much each result could change the conclusion. First, rebuild Kindnet with the agent metrics server enabled and rerun one tier, preferably 7 identities at 5,000 Pods per ReplicaSet; all six agent queries returned no samples, so no run has a λ_q , a μ , drop counts, or agent CPU and memory. Second, collect the `wait-for-gateway` init container’s `Latency`: lines, which every sandbox already logs and the harness did not keep; large values on the slow Pods implicate metadata convergence or the verdict path, small ones exonerate both and point at kubelet or the runtime. Third, equalize achieved Pod throughput or explain why the KNP cluster created 103–124 Pods per second against Cilium’s 302–328 on identical control-plane settings; a slower creator means a smaller per-node burst, which flatters the tail, so the closeness at 700 and 3,500 identities proves less than it appears to. Fourth, a matched pair at 35,000 identities, a Cilium raw-Pod mesh run or a KNP run with one Pod per ReplicaSet. Fifth, repeat trials; with one sample per tier the 0.5 s and 2.4 s differences are indistinguishable from noise. Sixth, a `cilium-dbg bpf policy get dump` and the effective `cilium-config` from a gateway node, so that Figure 2 rests on a measurement rather than the indirect argument of Section 6. Seventh, a single-node connection microbenchmark with a denied-traffic mix, run into overload in both queue modes, since the capacity boundary of Section 3 is an overload point nobody has measured. Last, vary label sharing

and workload freshness independently at fixed live-Pod count, to separate identity dissemination from Pod churn.

The disconfirming outcomes are easy to state. Agent saturation under burst, or `Latency`: lines that account for the tail, would attribute the 151 s and 421 s tails to deferred evaluation rather than a confound; equalized throughput that widens the gap at 700 and 3,500 identities would remove the one place where KNP currently looks competitive; a factored-kernel baseline completing the 35,000-identity mesh cheaply would reduce the capacity claim to a statement about one representation; and a denied-traffic microbenchmark that fills the 1,024-packet queue at a connection rate a real tenant could produce would turn the overflow boundary from a constant in the source into a measured limit.

8.3 Limits of the Present Claim

The design does not promise unlimited identities, constant total memory, zero startup delay, or resource use independent of topology. It removes one eager cross-product by keeping semantic evaluation in userspace and caching the outcome per connection, a good trade only where the admission and consistency costs are acceptable. The KNP mesh sweep supports a capacity statement: a tier the Cilium harness excluded on predicted per-endpoint map demand completes under KNP with all 35,000 Pods Running and none stranded, and going from 700 to 3,500 identities does not raise its P99 (5.331 s to 2.791 s). It does not support an admission-latency statement: KNP is slower than Cilium at every matched tier, by 2.4 s and 0.5 s where it is close and by a factor of 22 at 7 identities (151.479 s against 6.767 s); the raw-Pod tier records 421.410 s with two startup-SLO failures; achieved Pod throughput is 2.5–3× lower for reasons nobody recorded; every tier is one trial; and the agent metrics are empty. Enforcement is verified only on the positive path, by the gateway gate every sandbox passes; the negative path was checked once per cluster by `scripts/verify-netpol.sh`, unarchived.

Two further caveats. Results are version-specific on both sides: an archived Cilium v1.18.6 run stranded 19 Pods at 700 identities on a policy-cache race fixed in v1.20.0 [1], and the KNP figures come from one Kindnet v1.0.1 build with no metrics port and unrecorded agent flags. And the semantics enforced are looser than the discussion sometimes implies: NetworkPolicy leaves the effect of a policy update on established connections implementation-defined [19], so caching an accepted verdict for the life of the connection is conforming on its own, strict mode’s conntrack scan is a choice KNP makes rather than a requirement the API imposes, and an engine that does revoke is equally within the API. Comparing revocation cost compares design choices, not conformance.

9 Conclusion

NetworkPolicy at scale is a joint problem of representation, placement, and synchronization, and the three cannot be optimized one at a time. A fast kernel lookup does not make workload admission cheap, and a flat latency curve does not mean the policy state has no ceiling. NFQUEUE, conntrack, and NRI permit a different decomposition: interpret the policy when communication is attempted, keep the accepted decision in the kernel’s connection state, and learn local Pod addresses from the runtime rather than from the API server. None of these mechanisms is new, and the decomposition inherits the known weakness of reactive processing: its cost concentrates in the miss bursts.

The archived experiments on 720 workers and 35,000 Pods show both sides. Cilium’s completed sweeps hold low post-scheduling latency and flat worker CPU across their feasible region, halve achieved throughput at 35,000 ReplicaSets, exclude the 35,000-identity bidirectional mesh on predicted map capacity, and stranded 5,694 Pods in one hub-and-spoke raw-Pod burst. The KNP sweep completes the excluded tier with every Pod Running, stays within 2.4 s of Cilium’s P99 at 700 and 3,500 identities, and records 151 s and 421 s tails in its two burst tiers that the artifacts cannot attribute, at a Pod-creation rate 2.5–3× below Cilium’s that they cannot explain either. Because every sandbox gates on a permitted TCP connection to the gateway before it reports Running, these latencies contain the first permitted cross-node communication; that makes them a more honest measure than container startup, and it makes the unexplained tails the most important open item, since they are tails on the very quantity the design claims to improve. Completing the comparison requires the measurements of Table 4, beginning with an agent that exposes its metrics, the init-container latencies the sandboxes already record, and a second trial of the matched tiers.

References

- [1] Cilium contributors. Pods stuck in Init when policy map is full. Cilium issue 7515, closed by pull request 44900. <https://github.com/cilium/cilium/issues/7515>. Accessed September 17, 2026; the fix is reported in the v1.20.0 release notes.
- [2] Michael Dalton, David Schultz, Jacob Adriaens, Ahsan Arefin, Anshuman Gupta, Brian Fahs, Dima Rubinstein, Enrique Cauich Zermeno, Erik Rubow, James Alexander Docauer, Jesse Alpert, Jing Ai, Jon Olson, Kevin DeCaboooter, Marc de Kruijf, Nan Hua, Nathan Lewis, Nikhil Kasinadhuni, Riccardo Crepaldi, Srinivas Krishnan, Subbaiah Venkata, Yossi Richter, Uday Naik, and Amin Vahdat. Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, 2018. <https://www.usenix.org/conference/nsdi18/presentation/dalton>.
- [3] jeevanragula and Kubernetes contributors. Unable to set external store in client-go. Kubernetes issue 114659, 2022. <https://github.com/kubernetes/kubernetes/issues/114659>. Opened December 22, 2022; accessed September 16, 2026.
- [4] Teemu Koponen, Keith Amidon, Peter Balland, Martín Casado, Anupam Chanda, Bryan Fulton, Igor Ganichev, Jesse Gross, Natasha Gude, Paul Ingram, Ethan Jackson, Andrew Lambeth, Romain Lenglet, Shih-Hao Li, Amar Padmanabhan, Justin Pettit, Ben Pfaff, Rajiv Ramanathan, Scott Shenker, Alan Shieh, Jeremy Stribling, Pankaj Thakkar, Dan Wendlandt, Alexander Yip, and Ronghua Zhang. Network virtualization in multi-tenant datacenters. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. USENIX Association, 2014. <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/koponen>.
- [5] Shengkai Lin, Shizhen Zhao, Peirui Cao, Xinchu Han, Quan Tian, Wenfeng Liu, Qi Wu, Donghai Han, and Xinbing Wang. ONCache: A cache-based low-overhead container overlay network. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025. <https://www.usenix.org/conference/nsdi25/presentation/lin-shengkai>.
- [6] Sathiya Kumaran Mani, Kevin Hsieh, Santiago Segarra, Ranveer Chandra, Yajie Zhou, and Srikanth Kandula. Securing public cloud networks with efficient role-based micro-segmentation. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 2025. <https://www.usenix.org/conference/nsdi25/presentation/mani>.
- [7] Antonio Ojea. Optimize pod IP and metadata distribution for large clusters using kube-ip-tracker. kube-network-policies pull request 190, 2025. <https://github.com/kubernetes-sigs/kube-network-policies/pull/190>. Accessed September 16, 2026; implementation history, not an independent evaluation.
- [8] Antonio Ojea and Marek Siarkowicz. Add scalability tests for ipcache. kube-network-policies pull request 218, discussion thread, 2025. <https://github.com/kubernetes-sigs/kube-network-policies/pull/218>. Accessed September 17, 2026; single-client integration test logs, not an independent evaluation.
- [9] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, Keith Amidon, and Martín Casado. The design and implementation of Open vSwitch. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, 2015. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pfaff>.
- [10] Sheng Qi, Zhiqian Zhang, Xuanzhe Liu, and Xin Jin. KUBEDIRECT: Unleashing the full power of the cluster manager for serverless computing. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 2026. <https://www.usenix.org/conference/nsdi26/presentation/qi>.
- [11] spikecurtis and Kubernetes contributors. Pod.Status.PodIP not updated during postStart lifecycle hook. Kubernetes issue 85966, 2019. <https://github.com/kubernetes/kubernetes/issues/85966>. Opened December 5, 2019; closed January 30, 2025; accessed September 16, 2026.
- [12] Phil Sutter. Benchmarking nftables. Red Hat Developer, 2017. <https://developers.redhat.com/blog/2017/04/11/benchmarking-nftables>. Published April 11, 2017; accessed September 16, 2026.
- [13] The Antrea Authors. Antrea OVS pipeline. Antrea design documentation. <https://antrea.io/docs/main/docs/design/ovs-pipeline/>. Accessed September 17, 2026; rolling documentation.
- [14] The Cilium Authors. cilium-agent command reference. Cilium documentation, `-bpf-policy-map-max`.

- <https://docs.cilium.io/en/stable/cmdref/cilium-agent/>. Accessed September 17, 2026. The v1.20.1 source (pkg/maps/policymap/cell.go, pkg/option/config.go) sets the default to 16,384 and clamps the flag to [256, 65,536]; neither is a record of the benchmark configuration.
- [15] The Cilium Authors. Security identities. Cilium documentation. <https://docs.cilium.io/en/stable/internals/security-identities/>. Accessed September 16, 2026; not a benchmark version record.
 - [16] The Kubernetes Authors. API priority and fairness. Kubernetes documentation. <https://kubernetes.io/docs/concepts/cluster-administration/flow-control/>. Accessed September 16, 2026.
 - [17] The Kubernetes Authors. kube-network-policies: BoltStore implementation. Source code, pkg/ipcache/bbolt.go. <https://github.com/kubernetes-sigs/kube-network-policies/blob/main/pkg/ipcache/bbolt.go>. Local source inspected September 16, 2026; verify pinned commit accessibility before publication.
 - [18] The Kubernetes Authors. kube-network-policies testing notes: ApacheBench microbenchmark. Repository documentation, docs/testing/README.md. <https://github.com/kubernetes-sigs/kube-network-policies/blob/main/docs/testing/README.md>. Local copy inspected at revision 9984dcd; hardware and software versions are not recorded.
 - [19] The Kubernetes Authors. Network policies. Kubernetes documentation. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. Accessed September 16, 2026; see Pod lifecycle and existing connections.
 - [20] The Linux Kernel Contributors. BPF maps. Linux kernel documentation. <https://docs.kernel.org/bpf/maps.html>. Accessed September 16, 2026.
 - [21] The Netfilter Project. libnetfilter_queue: Queue handling. API documentation, version 1.0.5. https://netfilter.org/projects/libnetfilter_queue/doxygen/html/group__Queue.html. Accessed September 16, 2026.
 - [22] The Open vSwitch Project. ovs-fields(7): Conjunctive match fields. Open vSwitch manual, conj_id. <https://www.openvswitch.org/support/dist-docs/ovs-fields.7.txt>. Accessed September 17, 2026.
 - [23] The OVN Project. ovn-nb(5): OVN north-bound database schema. OVN manual, ACL table. <https://www.ovn.org/support/dist-docs/ovn-nb.5.html>. Accessed September 16, 2026; rolling documentation.
 - [24] Tigera. Component architecture. Calico Open Source documentation, Felix and Typha. <https://docs.tigera.io/calico/latest/reference/architecture/overview>. Accessed September 17, 2026.
 - [25] Tigera. Typha overview. Calico Open Source documentation. <https://docs.tigera.io/calico/latest/reference/typha/overview>. Accessed September 16, 2026.
 - [26] Dan Winship. NFTables mode for kube-proxy. Kubernetes Blog, 2025. <https://kubernetes.io/blog/2025/02/28/nftables-kube-proxy/>. Published February 28, 2025; accessed September 16, 2026.

A Evidence Provenance

This appendix is the provenance guide for the non-submission exercise: for every table, figure, and quoted count in Section 6 it names the file that was read and the key that was taken from it. Paths are relative to the benchmark repository unless stated otherwise.

Archived numeric results. The extractor is `submission/analyze.mjs`. The current extraction covers all 66 benchmark run directories and 59 startup reports and is recorded in `submission/upstream/inventory.json`, `submission/upstream/startup.csv`, and `submission/upstream/extracted-results.md`. Startup percentiles are the `Perc50`, `Perc90`, and `Perc99` fields (in milliseconds) of the `dataItems` entries whose `labels.Metric` is `create_to_schedule`, `schedule_to_run`, or `pod_startup` in each run's `PodStartupLatency_TotalPodStartupLatency_*.json`. Failure counts are `<failure>` elements in each run's `junit.xml`. Run directory names do not establish tested software versions; those come from `agent-cilium.k8s.local.yaml`, `agent-netpol.k8s.local.yaml`, and `scripts/create-netpol.sh`, and no image digest is archived.

Table 1 and Figure 2. The Cilium open-gateway rows are the four directories under `artifacts_pods/cilium/identity-sweep/` (`7-identities-5000ppr`, `700-identities-50ppr`, `3500-identities-10ppr`, `35000-identities-1ppr`). The hub-and-spoke rows are `artifacts_pods/cilium/microsegmentation/` `tier1-7-identities-5000ppr`, `tier2-700-identities-50ppr`, `tier3-3500-identities-10ppr`, and `tier4-35000-identities-1ppr-rs`. The Cilium mesh rows are `artifacts_pods/cilium/mesh-sweep/tier1-7-identities-5000ppr`, `tier2-700-identities-50ppr`, and `tier3-3500-identities-10ppr`; their sibling `*-initial` directories hold hook logs only and are the three hook-only attempts of Section 6.1. The KNP mesh rows are `artifacts_pods/kindnet/mesh-sweep/tier1-7-identities-5000ppr`, `tier2-700-identities-50ppr`, and `tier4-35000-identities-rawpods`. Every KNP tier's `generatedConfig_agentic-sandboxes.yaml` sets `SandboxPeerMode: mesh`; the Cilium raw-Pod directory `artifacts_pods/cilium/microsegmentation/tier4-35000-identities-rawpods` does not, which is why the two raw-Pod rows are not a matched pair. The KNP tier-4 `junit.xml` records the two startup-SLO failures. The

right-hand panel of Figure 2 is the model of Section 4, cross-checked against the guard in `scripts/run-mesh-sweep.sh` and the policy manifests `policy-scenarios/scenario-c-mesh-bidirectional.yaml`.

Table 3. The Cilium rows are the primary runs under `artifacts_pods/cilium/qps-50`, `qps-100`, `qps-200`, and `qps-500/verified-run`; the separate Cilium trial is `qps-500/repeat-run`. The Kindnet rows are `artifacts_pods/kindnet/baseline-sweep/qps-*`. The final tuning bundle is `artifacts_pods/kindnet/500qps-optimizations/5-v1.0.1-plus-nri-plus-apf-exempt/`; its four predecessors are the sibling directories 1- through 4-. The teardown-timeout diagnosis of the asterisked rows is read from the `WaitForOldPodsDeleted-Final` failure messages in the `junit.xml` files of the QPS 50 and 100 directories of both families.

Table 2: throughput, CPU, and API latency. Phase-1 Pods/s are the top-level `perc50` and `perc90` of `Phase1SchedulingThroughput_*.json` in each run directory (the tier-4 peak is its max). Worker CPU is read from `GenericPrometheusQuery Worker Node CPU Utilization_*.json` under `dataItems[0].data`: the table uses `mean_instantaneous_node_cpu_cores` and `p99_instantaneous_node_cpu_cores`, and the tier maxima quoted in Section 6.3 are `max_instantaneous_node_cpu_cores`. The Kindnet QPS-500 baseline row uses `mean_node` and `p99_node` from the July rate-based query and is flagged as such. Pod LIST P99 is the cluster-scoped pods LIST row of `APIResponsivenessPrometheus_simple_*.json`. The `max_kindnetd_cpu_cores` term in the saved CPU query has no samples in any run. No value in the table was transcribed from a Markdown summary.

Enforcement gate. The wait-for-gateway init container is defined in `manifests/agent-c-sandbox/manifests/sandbox-pod.yaml` and `sandbox-replicaset.yaml`. The policies that make the gate a permitted-connection test are `global-sandbox-policy.yaml` (sandbox egress to group: gateway, TCP 80) and `gateway-policy.yaml` (gateway ingress from group: sandbox), with `default-deny-policy.yaml` applied in every archived run. The one-shot negative check is `scripts/verify-netpol.sh`; no output of it is archived, and no run script calls it.

KNP agent queries. The six `GenericPrometheusQuery Kube Network Policies *` files (CPU Usage, Memory Usage, Packet Processing Time, Packet Verdicts Rate, Queue Drops Rate, User Drops Rate) and the `GenericPrometheusQuery Kubelet CNI Plugin`

Operations Latency file in all four KNP mesh tiers contain "dataItems": null. The queries are defined in manifests/agent-sandbox/measurements/knp-measurements.yaml; scripts/query-kindnet-metrics.sh records that the deployed Kindnet build exposes no metrics port. The Sandbox Pod Density Per Node query in tier 4 reports max_pods_per_node of 23,789, which is almost certainly pending Pods bucketed under an empty node label; it is not a real node and is not cited.

Archived v1.18.6 attempt. artifacts_pods/archive/v1.18.6-identity-runs/ holds a complete 7-identity run and a 700-identity directory containing only cl2-metadata.json and the generated configuration. The 34,981/19 counts and the issue-7515 attribution are from benchmark_results.md, not from artifacts; no logs from the attempt were retained.

Outcome records. Revision 0442dce adds artifacts_pods/cilium/microsegmentation/tier4-35000-identities-rawpods/README.md, the source of the 29,306/5,694 completion counts and the stated HTTP 429 and identity-exhaustion diagnosis. Revision f998224 adds artifacts_pods/cilium/mesh-sweep/README.md, documenting the unexecuted Cilium mesh tier. Revision 5b152f2 adds artifacts_pods/kindnet/mesh-sweep/: three ReplicaSet-paced KNP tiers matched to the Cilium mesh generator and one raw-Pod tier. Two statements in that directory's README.md are not supported by its own artifacts and are not used here: "scheduler placed all pods in under 50 seconds" (the archived create_to_schedule P99 is 106.691 s) and "0 CNI errors" (the CNI-latency query returned no samples, so there is no metric behind the count). The README also omits the tier-4 JUnit SLO failures and the tier-1 latency. The 65,536 map limit is documented only in the header comment of scripts/run-mesh-sweep.sh; the cilium-config ConfigMap it reads at run time is not archived.

Original charts. The input figure figures/fig2_identity_cardinality_sweep.png mixes measured quantiles, Kindnet tier values with no located source run, and a failure bar; figures/fig2_topological_scaling_p99.png contains a 94.3-second overflow value and a kernel-set description of KNP that the audited evidence does not support. Both are preserved as author input and neither is reproduced here. Figure 2 is typeset directly from the audited quantiles and the explicitly conditional capacity model.

Implementation. Paths in the companion kube-network-policies repository, inspected at 9984dcd, are pkg/dataplane/controller.go (queue configuration, overflow

mode, postrouting and input chains), pkg/cmd/cmd.go (flag defaults), pkg/networkpolicy/networkpolicy.go (nil-peer selector semantics, managed-IP set), plugins/iptracker/iptracker_networkpolicy.go (divert-all), pkg/podinfo/nri/resolver.go, pkg/ipcache/{client,bbolt,lru}.go, and docs/testing/README.md with its two PNG charts. Benchmark-side deployment facts are from manifests/kube-network-policies-install.yaml and scripts/create-netpol.sh. Source behavior has not been equated to the benchmark binaries. submission/evidence-audit.md records the claim-level audit and submission/author-guide.md the prioritized remaining checks.